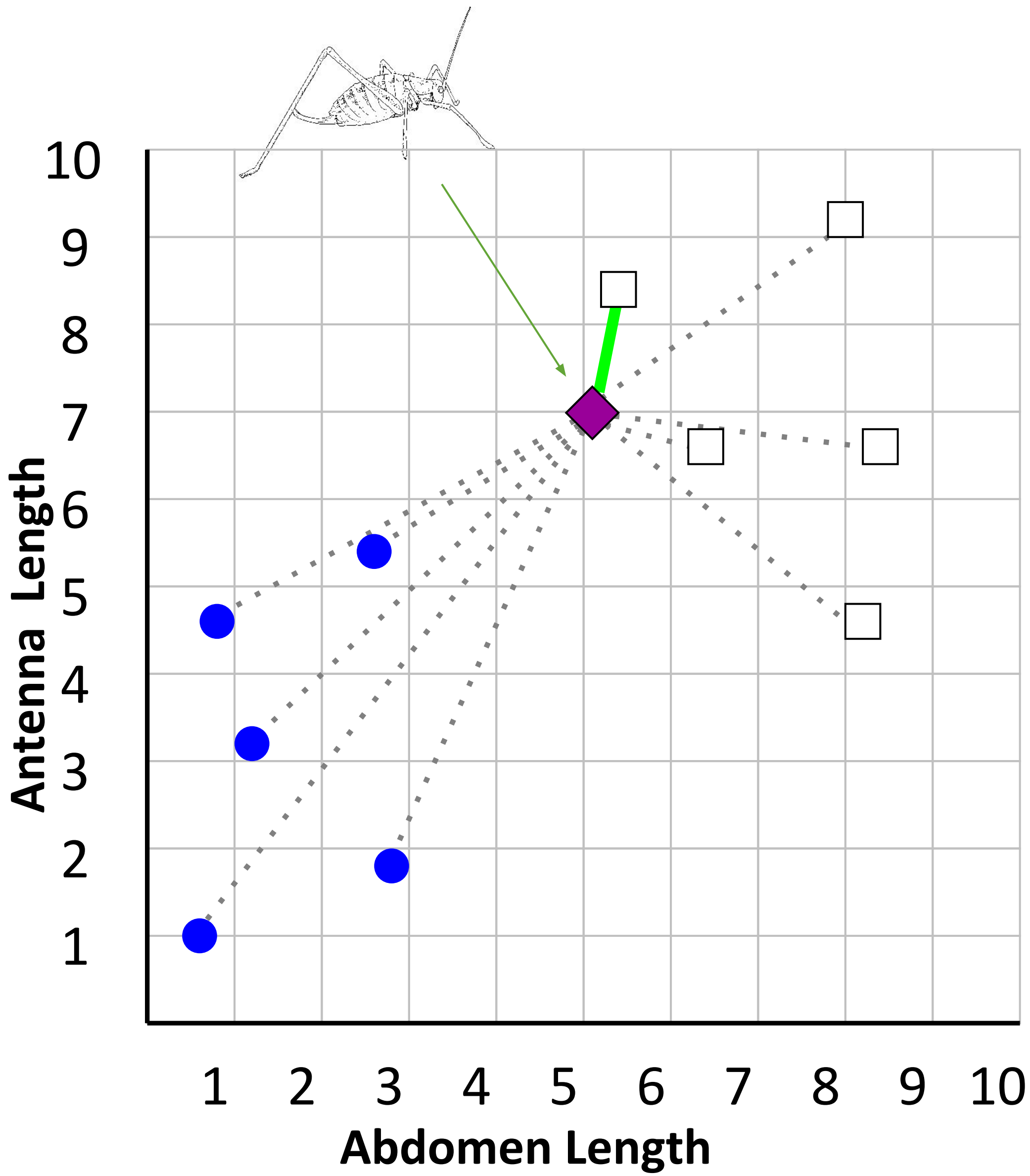
k-Nearest Neighbors

Nearest Neighbor Example



If the nearest instance to the previously unseen instance is a **Katydid**

class is **Katydid**

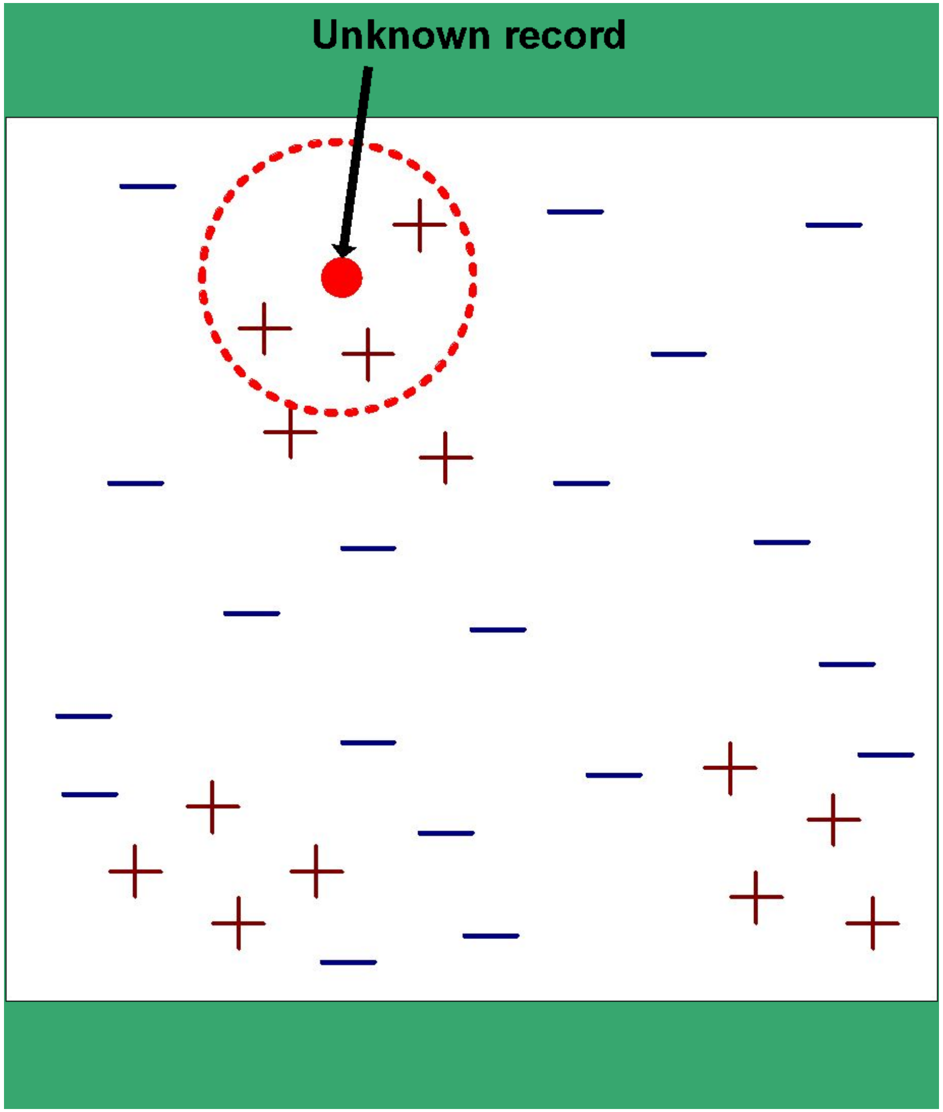
else

class is **Grasshopper**

□ **Katydids**

● **Grasshoppers**

What is Needed for Nearest Neighbor



Three things are needed
- Set of stored training records
- Distance metric
- # of nearest neighbors *k*

To classify unknown record
- Compute distance to every training record
- Identify *k* nearest neighbors
- Determine classification using the k nearest neighbors
  - Using majority vote or weighted vote

Nearest Neighbor is Lazy Learner

Most Learners are Eager

- Work is done up-front
  - Generate an explicit description of target function
  - That is build a model from the training data

Lazy Learner

- Does not build a model: work is deferred
- Learning phase
  - Just store the training data
- Testing Phase
  - Essentially all work is done when classifying the example.
  - No explicit model but rather implicit in the data and metrics

Assessing Similarity Not Easy

Computer would say these two images are extremely similar, but humans notice flaws with one Homer (on right)

These are superficially very different from first one above, but humans see the similarity

Issues with Different Scales

Examples below described by 3 numeric features

John:
Age = 35
Income = 35,000
No. of credit cards = 3

Rachel:
Age = 22
Income = 50,000
No. of credit cards = 2

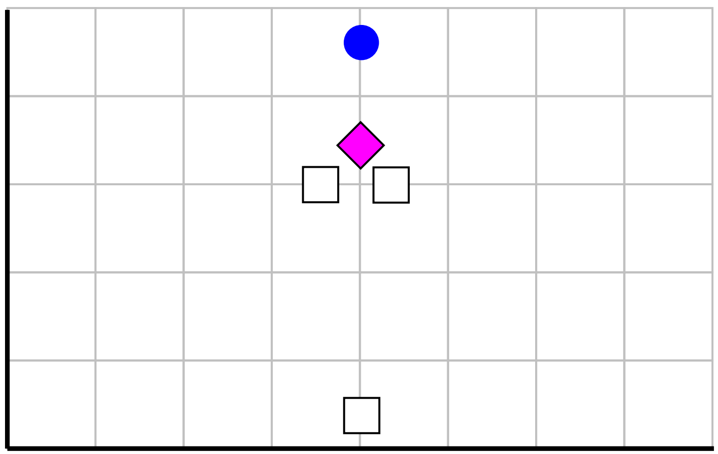
“Closeness” defined in terms of the distance

- Euclidean distance: square root of sum of the squared differences
- Distance(John, Rachel) = sqrt$[(35-22)^2+(35K-50K)^2+(3-2)^2]$

Problem: income dominates due to scale

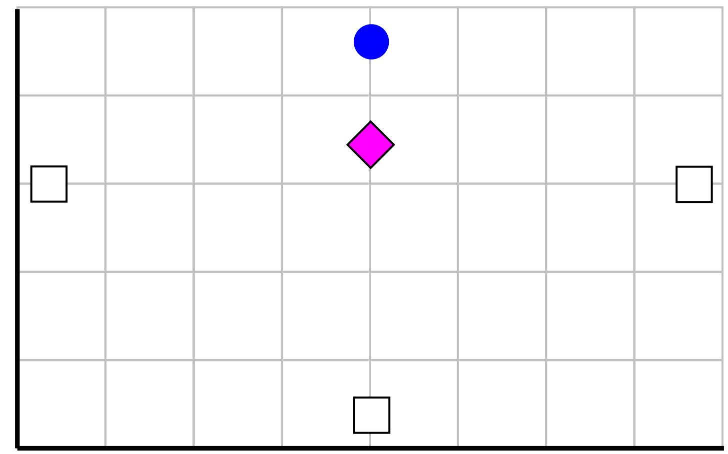
- Solution: rescale features to uniform range

Issues with Different Scales



X axis measured in **centimeters**
Y axis measure in dollars

The nearest neighbor to the **pink** unknown instance is □



X axis measured in **millimeters**
Y axis measure in dollars

The nearest neighbor to the **pink** unknown instance is **blue**.

Use z-normalization so feature values have a mean of zero and a standard deviation of 1. Can use this formula: X = (X – mean(X))/std(X)

Irrelevant Features

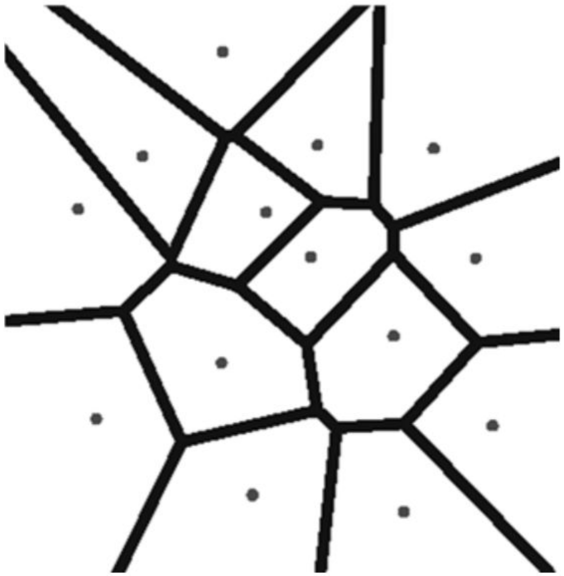
If each example described by 20 attributes but only 2 are relevant

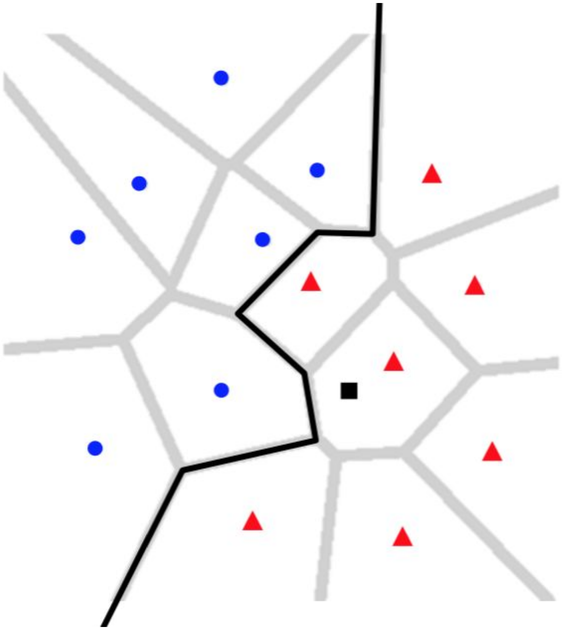
- Examples with identical values for the 2 attributes may still be distant in 20-dimensional instance space

How to mitigate irrelevant features?

- Use more training instances
  - Harder to obscure patterns
- Ask an expert which features are irrelevant and drop
- Use statistical tests (prune irrelevant features)
- Search feature subsets

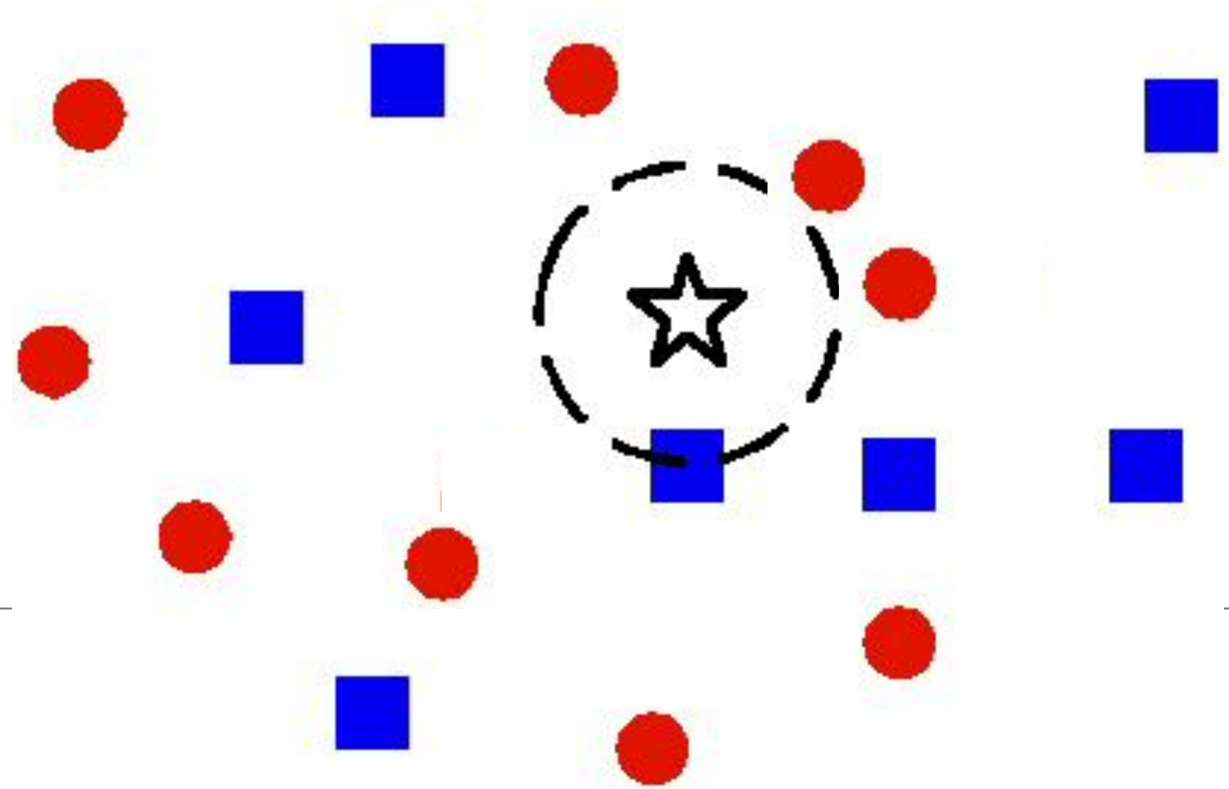
What happens for k = 1?



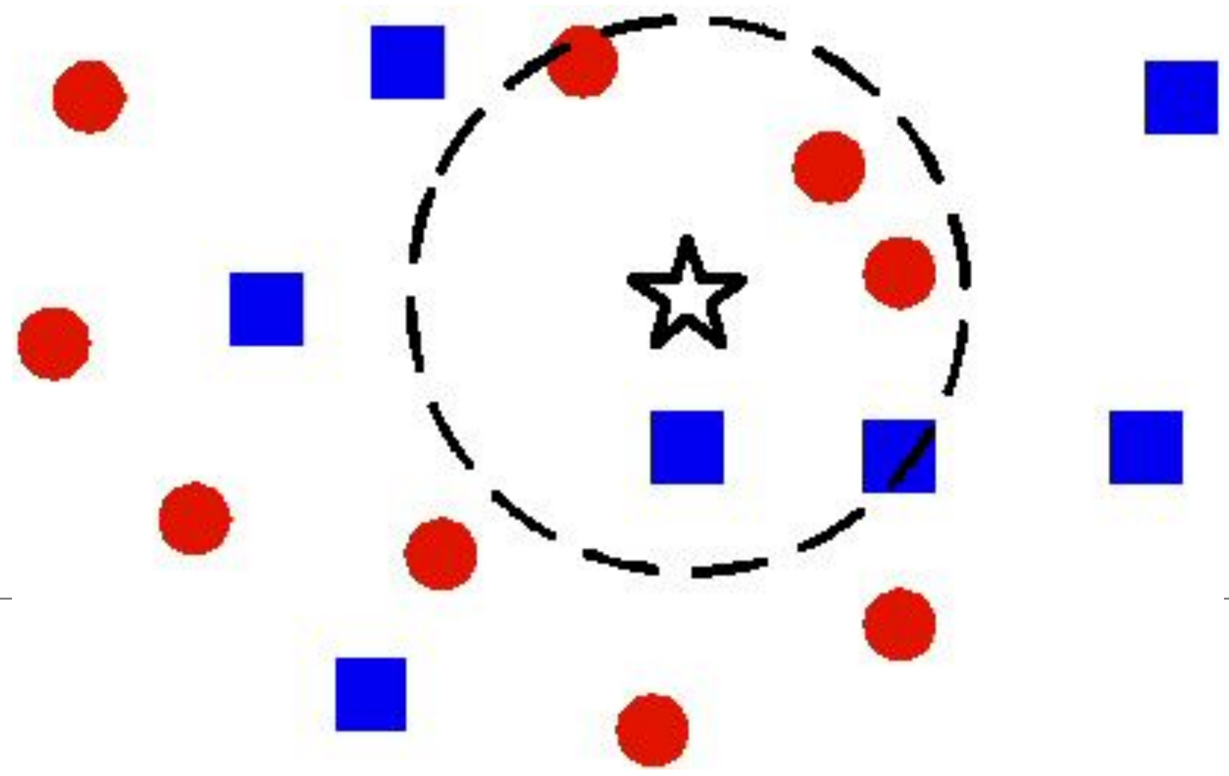


- We obtain a Voronoi diagram:
- The space is partitioned into regions
- The separating borders pass through areas where the distances between training sample pairs are equal
- The separating borders are nonlinear

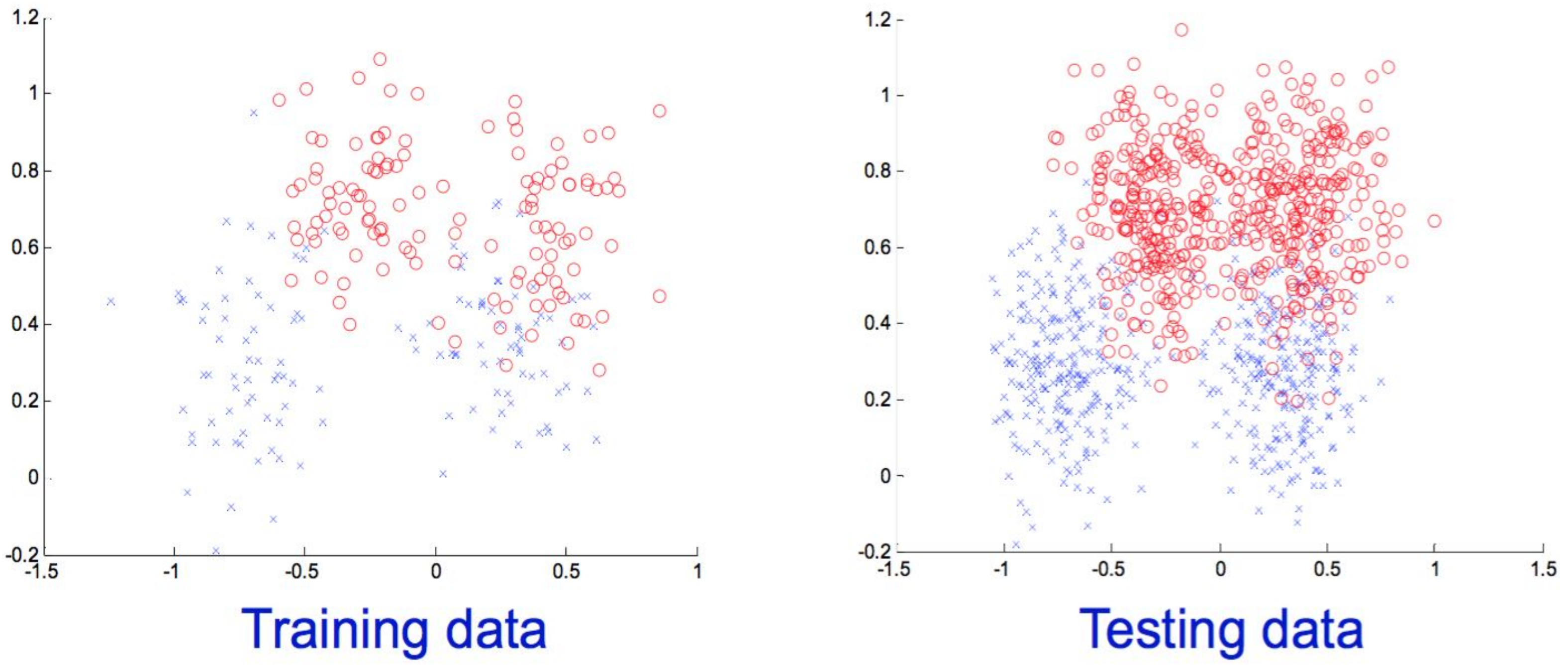
1-NN versus k-NN



1-NN versus k-NN



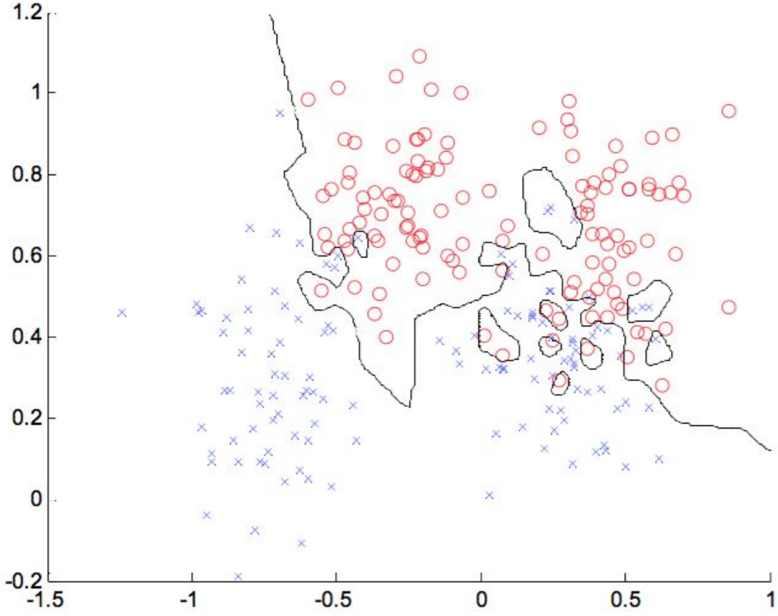
The underlying hypothesis of k-NN



- The training data and the test data are sampled from the same distribution
- Becomes unlikely for a representative pattern in the training set to be absent in the test set

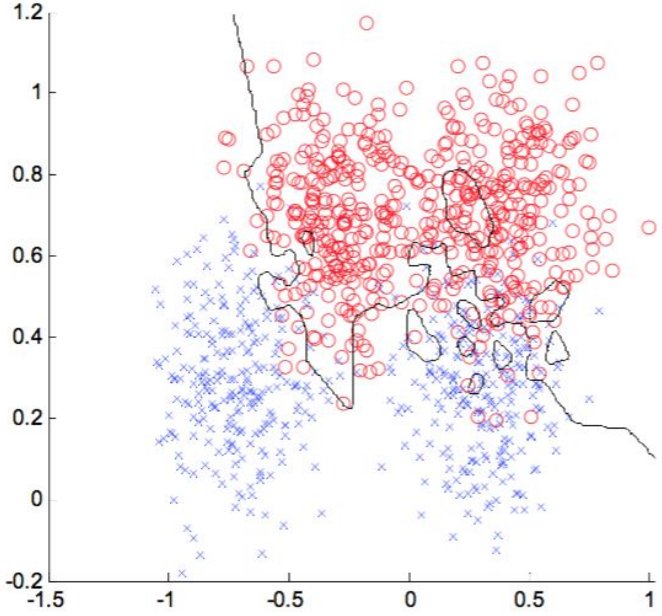
What happens for different values of k?

Training data



error = 0.0

Testing data

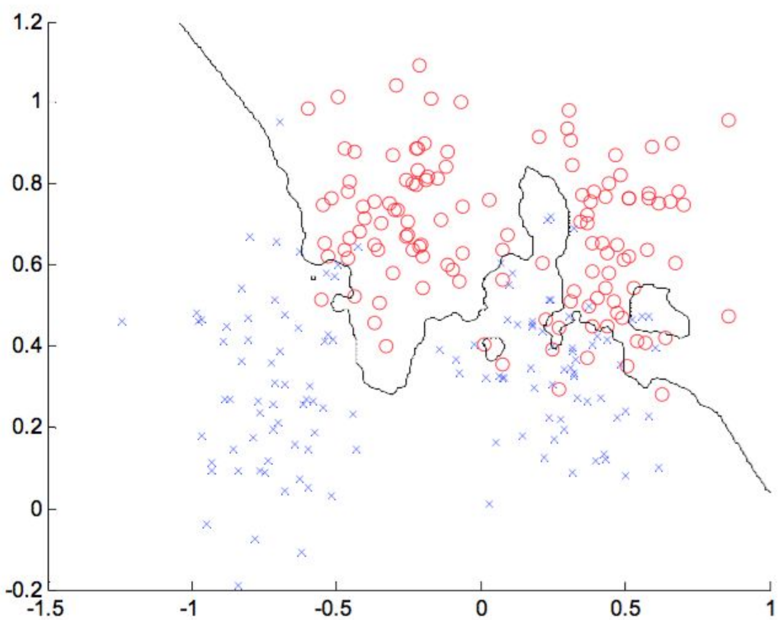


error = 0.15

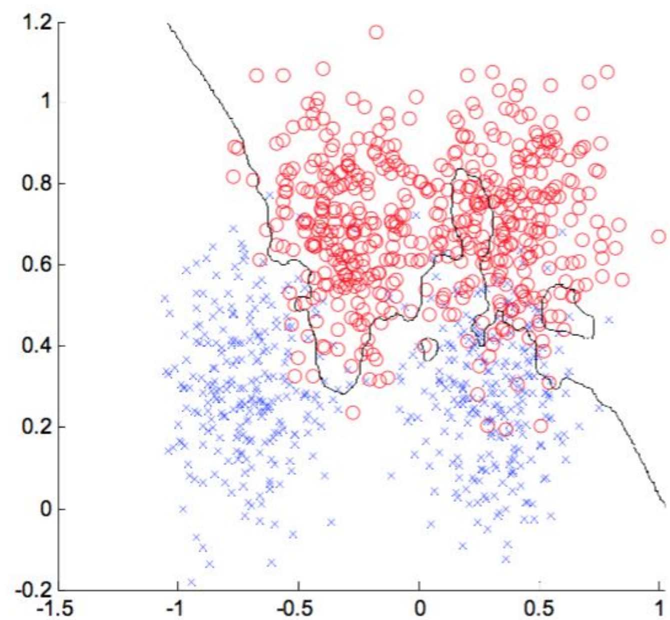
- k = 1

What happens for different values of k?

Training data



Testing data



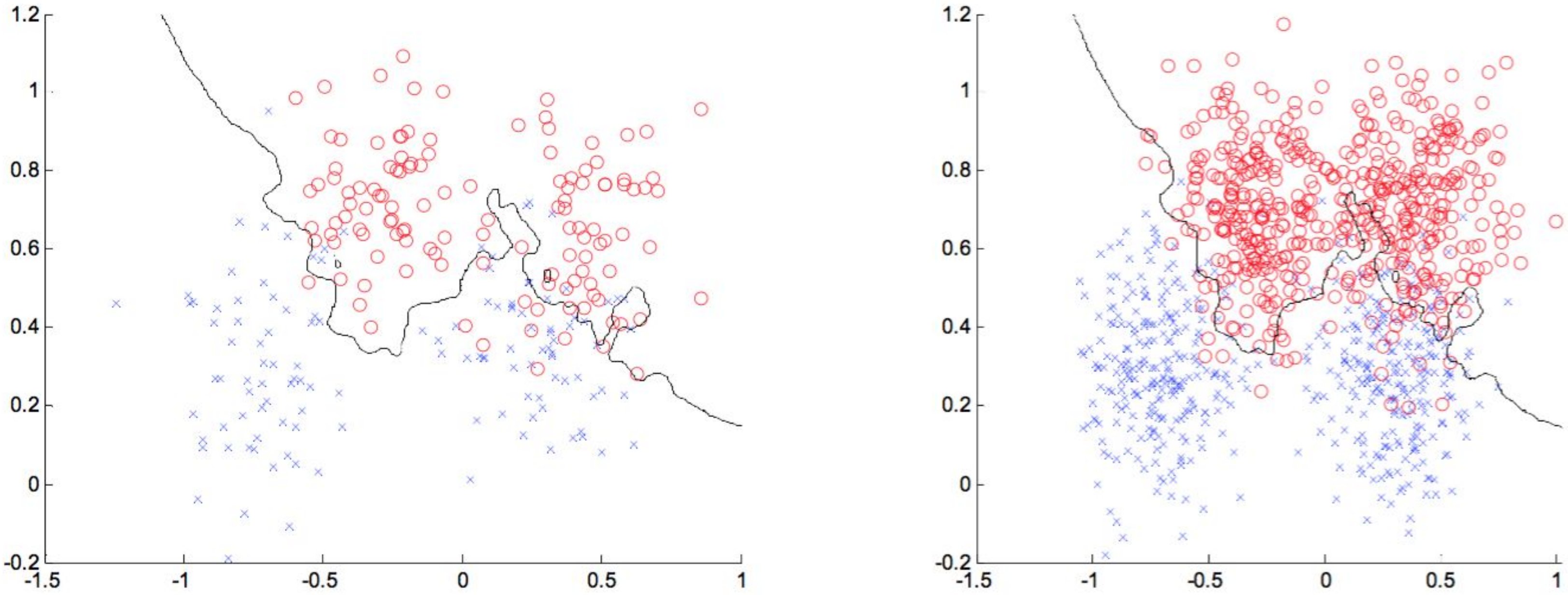
- k = 3

error = 0.0760

error = 0.1340

What happens for different values of k?

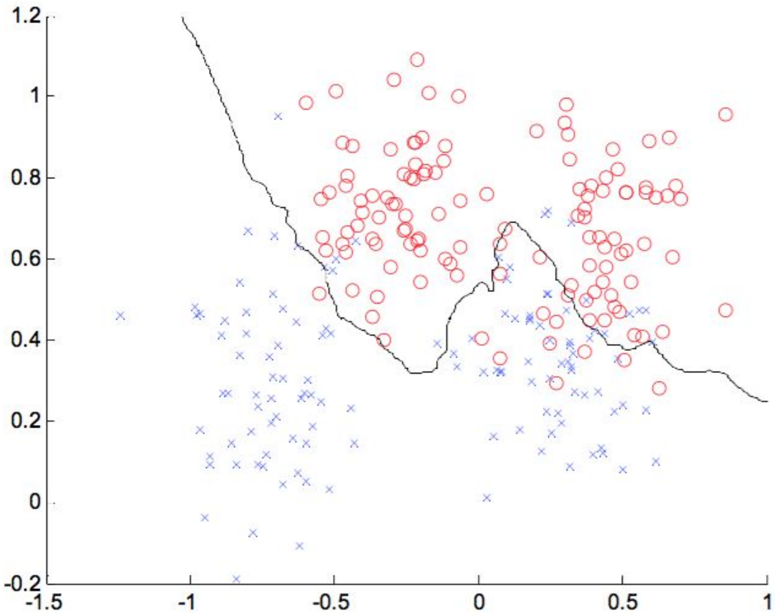
Training data　　　　　　　　Testing data



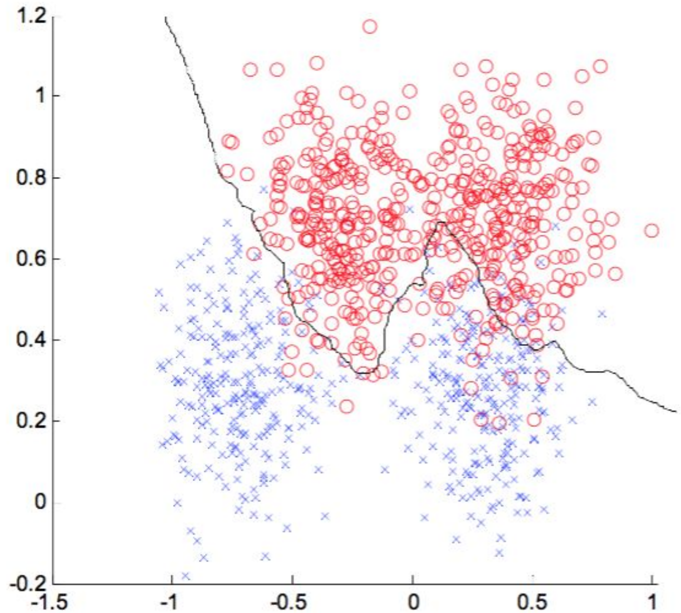
- k = 7　　error = 0.1320　　　　　error = 0.1110

What happens for different values of k?

Training data



Testing data



- k = 21

error = 0.1120

error = 0.0920

What do we need for a memory-based classifier?

- A DISTANCE FUNCTION:
  - **The Euclidean distance**
  - **The Edit (Levenstein) distance**
  - **The Hamming distance**
- HOW MANY NEIGHBORS SHOULD WE CONSIDER?
- HOW DO WE "TRAIN" THE MODEL ON THE EXAMPLES FROM THE VICINITY?

Let’s consider a particular 1-NN

The Euclidean distance (L_2)

- For the vectors $x = (5, 1,3,0)$ and $y = (2,1,4,1)$, we have:

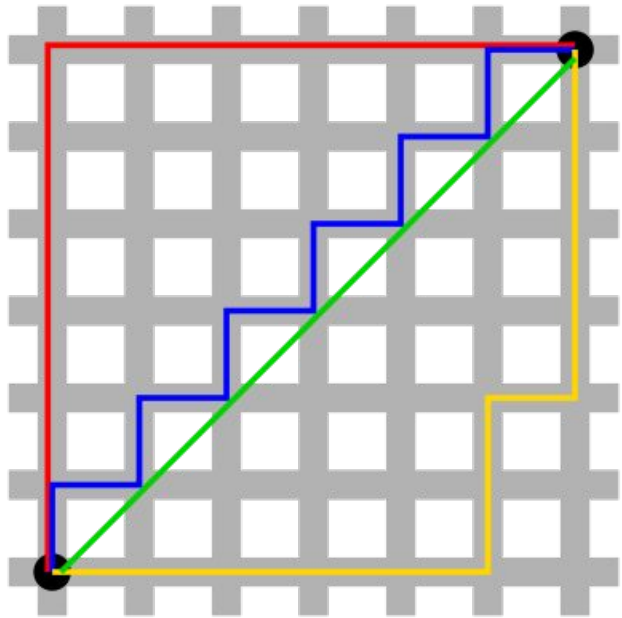
$$d_{L_2}(x, y) = \sqrt{(x_1 - y_1)^2 + \cdots + (x_n - y_n)^2}$$

$$= \sqrt{(5-2)^2 + (1-1)^2 + (3-4)^2 + (0-1)^2}$$

$$= \sqrt{9+1+1} = \sqrt{11}$$

$$\cong 3.32$$

The Manhattan distance (L_1)



- For the vectors $x = (5, 1,3,0)$ and $y = (2,1,4,1)$, we have:

$$d_{L_1}(x, y) = |x_1 - y_1| + \cdots + |x_n - y_n|$$

$$= |5 - 2| + |1 - 1| + |3 - 4| + |0 - 1|$$

$$= 3 + 1 + 1 = 5$$

The Minkowski distance (L_p)

- For the vectors $x = (x_1, \ldots, x_n)$ and $y = (y_1, \ldots, y_n)$, we have:

$$d_{L_p}(x, y) = \sqrt[p]{|x_1 - y_1|^p + \cdots + |x_n - y_n|^p}$$

- The Minkowski distance is a generalization for the Euclidean distance ($p = 2$) and the Manhattan distance ($p = 1$)

- If $p < 1$, then $d_{L_{p<1}}$ is no longer a distance. The triangle inequality is violated for $x = (0,0)$, $y = (1,1)$ and $z = (0,1)$:

$$d_{L_{p<1}}(x, y) > d_{L_{p<1}}(x, z) + d_{L_{p<1}}(z, y)$$

The Hamming distance

- Useful, for instance, when the samples are represented by categorical features or when the samples are DNA sequences
- For the vectors $x = (A, G, T, C)$ and $y = (G, G, T, A)$, we have:

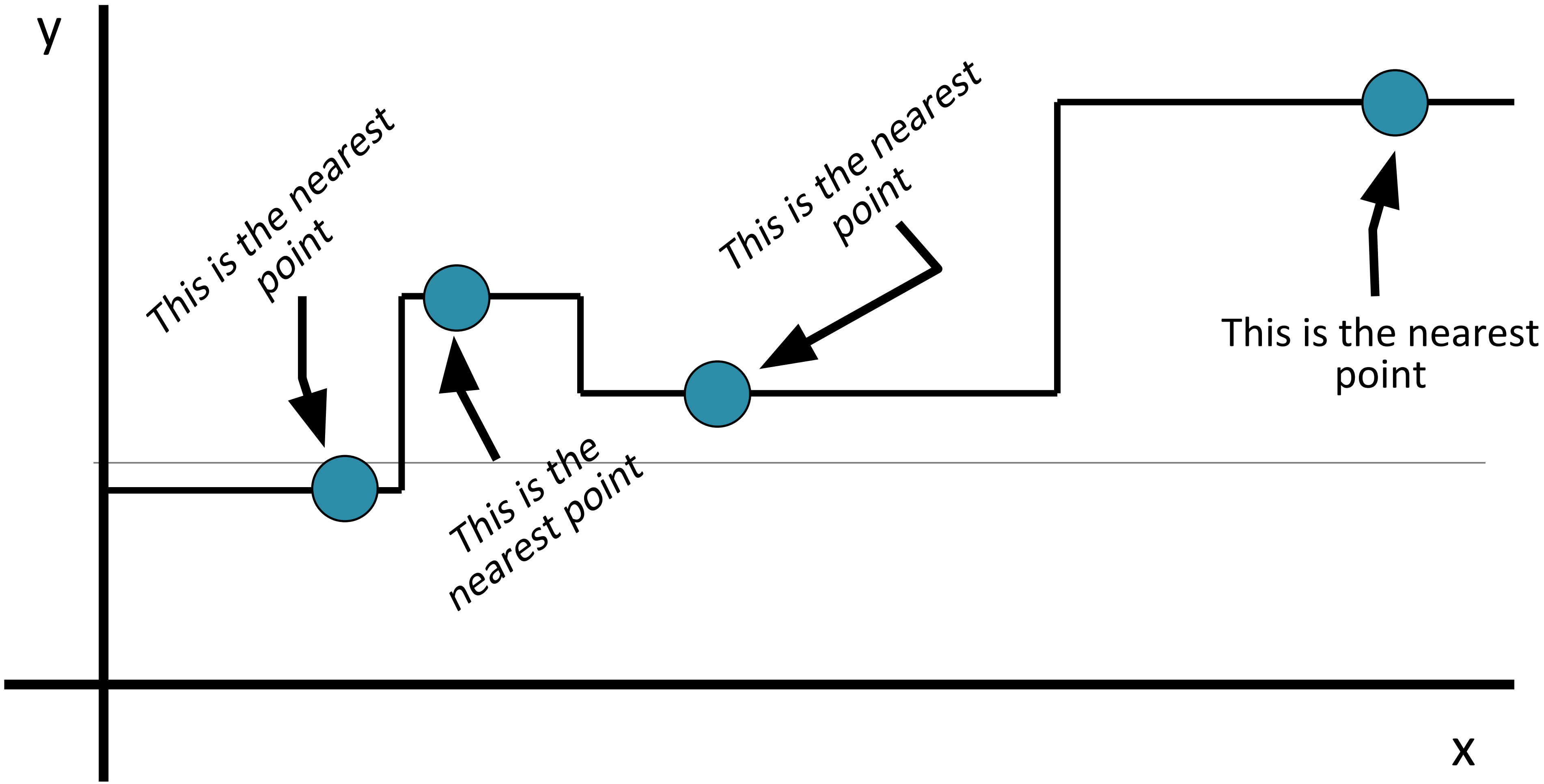
$$d_{Hamming}(x, y) = 1 + 0 + 0 + 1 = 2$$

- We are counting how many features (components) are different among the two vectors

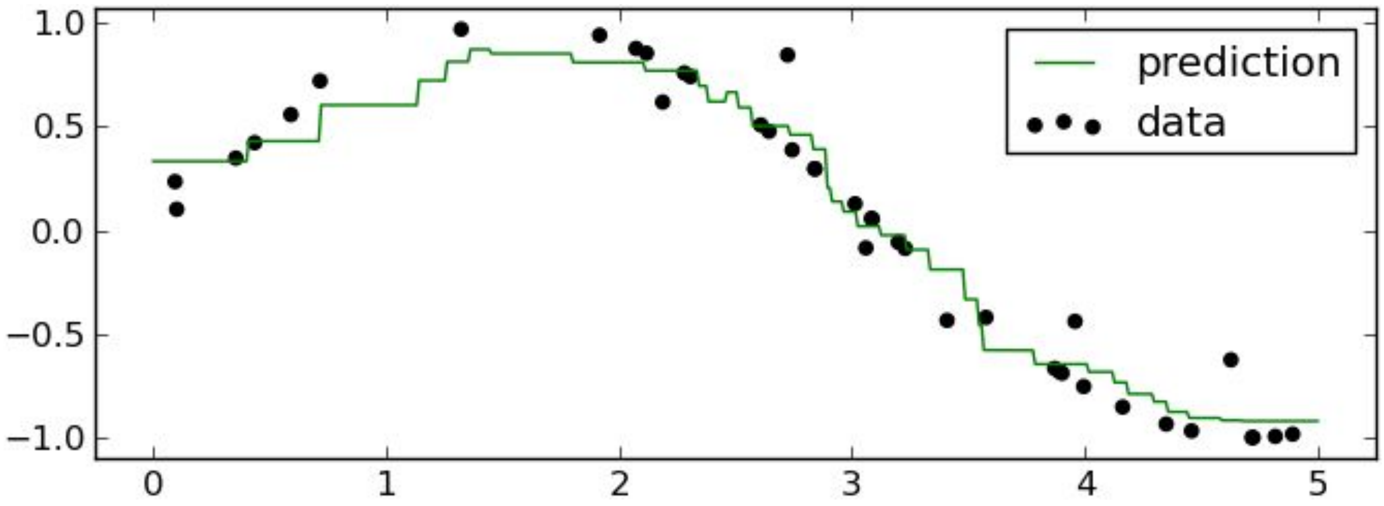
The Edit (Levenstein) distance

- Useful, for instance, when the samples are strings (text documents, DNA sequences) or temporal sequences (videos)
- The distance is given by the number of changes (insert, delete, replace) necessary to transform one object into the other
- For video sequences, we use Dynamic Time Warping (DTW)

1-NN for regression tasks



k-NN for regression tasks



- k-NN regression algorithm:

1) For each test sample x, we find the nearest k neighbors and their labels

2) The output is the mean of the labels of the k neighbors:

$$f(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^{K} y_i$$

Advantages and properties of k-NN

- k-NN is a very simple model
- Can be directly applied to multi-class problems
- The decision boundary is non-linear
- The quality of the results grows with the number of training samples
- We have a single parameter that requires tuning (k)
- The training error grows with k, but the decision boundary becomes smoother:

Disadvantages of k-NN

- What does nearest mean? We have to define a distance
- Is the Euclidean distance always the best choice?
- The computational cost is quite high: we need to store and pass through the whole training set during inference (at test time)

Thank You.

Acknowledgement:

Radu Ionescu, Prof. PhD.

Gary M. Weiss, CIS Dept, Fordham University